

Guía Conceptual del Lab: App de IA Generativa con Streamlit, Cloud Run y Gemini

1. El Propósito del Laboratorio: ¿Qué estamos resolviendo?

Muchos desarrolladores prueban modelos de lenguaje en Jupyter Notebooks o en Google AI Studio. Sin embargo, un prompt en una celda de código no es un producto.

El objetivo del lab es **llevar un prototipo de Inteligencia Artificial Generativa a producción real**:

1. Crear una interfaz visual interactiva para el usuario final (usando **Streamlit**).
2. Conectar esa interfaz con el modelo fundacional de Google (vía la **API de Gemini**).
3. Empaquetar el sistema para que funcione en cualquier entorno (usando contenedores **Docker**).
4. Publicarlo en internet con escalado automático y alta disponibilidad (usando **Cloud Run**).

2. La Arquitectura en 4 Capas (El "Behind the Scenes")

Antes de tocar comandos, los alumnos deben visualizar el flujo de datos:

- **Frontend / UI (Streamlit)**: Captura las entradas del usuario (ej. ingredientes, restricciones, opciones de menú o parámetros del cuento/receta) en la web y renderiza las respuestas.
- **Backend / Lógica (Python SDK)**: Toma esas variables, construye el prompt y realiza la llamada a la **API de Gemini**.
- **Empaquetado (Docker + Artifact Registry)**: Congela el código, el intérprete de Python y las librerías en una imagen inmutable guardada en un repositorio privado de Google Cloud.
- **Infraestructura Serverless (Cloud Run)**: Ejecuta ese contenedor en servidores administrados por Google, asignando una URL HTTPS pública y escalando de 0 a \$N\$ instancias según el tráfico.

3. Desglose Paso a Paso: Qué hacen y qué significa

Paso 1: Descarga del código fuente (**gcloud storage cp**)

Bash

```
gcloud storage cp -R gs://spl/spls/gsp1229/gemini-streamlit-cloudrun .
```

cd gemini-streamlit-cloudrun

- **¿Qué pasa aquí?** Se descarga una plantilla de código ya estructurada desde un bucket de almacenamiento (Cloud Storage).
- **Qué deben notar:** En la carpeta encontrarán principalmente tres archivos esenciales:
 - `app.py` (o `chef.py`): El código en Python que define la interfaz y la lógica de inferencia.
 - `requirements.txt`: Las dependencias externas (librerías como `streamlit`, `google-cloud-aiplatform` / SDK de GenAI).
 - `Dockerfile`: El archivo de instrucciones paso a paso para fabricar el contenedor.

Paso 2: Variables de entorno (**GOOGLE_CLOUD_PROJECT** y **REGION**)

Bash

```
GOOGLE_CLOUD_PROJECT="<ID_DEL_PROYECTO>"
```

```
GOOGLE_CLOUD_REGION="<REGION>"
```

- **¿Por qué son críticas?** La aplicación necesita saber a qué proyecto de Google Cloud facturar las consultas a Gemini y en qué centro de datos consultar el endpoint de la API. • Sin estas variables, el SDK de Google en Python no sabría qué infraestructura aprovisionar ni con qué permisos autenticarse.

Paso 3: Probar localmente en Cloud Shell (Modo Prototipado)

Antes de mandar nada a la nube, se ejecuta `streamlit run ...` en Cloud Shell.

- **Web Preview:** Cloud Shell redirige el puerto `8080` (o `8501`) a través de un proxy web seguro para que el alumno pueda probar su aplicación en una pestaña privada del navegador.
- **Lección clave:** Si falla aquí, fallará en producción. La regla de oro del desarrollador en la nube es verificar la lógica localmente antes de iniciar despliegues costosos.

Paso 4: Creación del Repositorio en Artifact Registry

Bash

```
gcloud artifacts repositories create [REPO_NAME] --repository-format=docker ...
```

- **¿Qué es Artifact Registry?** Es el "almacén de imágenes" de Google Cloud. • Para que un servicio en la nube (como Cloud Run) pueda levantar contenedores, primero debe tener un lugar seguro y centralizado desde donde descargar la imagen compilada.

Paso 5: Construcción de la Imagen con Cloud Build (**gcloud builds submit**)

Bash

```
gcloud builds submit --tag [REGION]-docker.pkg.dev/[PROJECT_ID]/[REPO]/[IMAGE]:[TAG]
```

- **¿Qué hace realmente este comando?**

1. Comprime el código y el **Dockerfile** de Cloud Shell.
2. Lo envía a **Cloud Build** (una máquina de compilación administrada en Google Cloud).
3. Ejecuta el **Dockerfile**: instala Python, descarga las librerías de **requirements.txt**, copia el código de Streamlit.
4. Genera la imagen Docker final y la sube automáticamente a Artifact Registry.

- **Pregunta clave para los alumnos:** *¿Por qué no lo construimos con **docker build** en Cloud Shell? Porque Cloud Build aprovecha la potencia de la nube de Google para compilar más rápido y sin consumir recursos de su máquina local.*

Paso 6: Despliegue en Cloud Run (gcloud run deploy**)**

Bash

```
gcloud run deploy [SERVICE_NAME] \  
  --image=[RUTA_DE_LA_IMAGEN] \  
  --region=[REGION] \  
  --allow-unauthenticated
```

- **¿Qué significa cada bandera?**

- **--image**: Qué versión empaquetada de nuestro software debe correr el servidor.
- **--allow-unauthenticated**: Hace que el servicio sea accesible por cualquier usuario en internet sin requerir autenticación previa en Google Cloud.

- **El resultado:** Google Cloud Run devuelve una **URL pública HTTPS**. Al ingresar a esa URL, Cloud Run enciende una instancia en milisegundos, inicia Streamlit y queda lista para recibir tráfico.

4. Preguntas para Evaluar la Comprensión

Al finalizar la práctica, los alumnos deben responder brevemente:

1. **¿Qué ventaja tiene desplegar la app en Cloud Run en lugar de dejarla corriendo en una máquina virtual de Compute Engine?**

Desplegar en Cloud Run ofrece un entorno "serverless" (sin servidor) totalmente administrado, lo que significa que escala automáticamente según el tráfico de tu aplicación, pudiendo reducirse a cero cuando no hay demanda, y solo te cobra por el tiempo exacto de procesamiento utilizado. En cambio, una máquina virtual en Compute Engine requiere mantenimiento manual del sistema operativo, gestión de la infraestructura y genera costos continuos aunque la aplicación esté inactiva.

2. **Si modificamos un texto en **app.py**, ¿los cambios se ven reflejados automáticamente en la URL de Cloud Run?**

No, los cambios no se reflejan automáticamente. Para que las modificaciones en el código fuente sean visibles en la URL pública, es necesario construir una nueva imagen de contenedor

(usualmente con Docker o Cloud Build) que incluya tu archivo app.py actualizado, y luego desplegar esa nueva revisión en el servicio de Cloud Run.

3. ¿Cómo se comunica la app de Streamlit con Gemini?

La aplicación de Streamlit se comunica con Gemini a través de su API (generalmente mediante Vertex AI o Google AI Studio) utilizando una biblioteca cliente o SDK de Python. Cuando un usuario ingresa información en la interfaz de Streamlit, la app empaqueta esos datos y envía una solicitud HTTP o de API al modelo de Gemini, el cual procesa la información y devuelve una respuesta que Streamlit finalmente renderiza en la pantalla.

Lab:

https://www.skills.google/focuses/85991?catalog_rank=%7B%22rank%22%3A12%2C%22num_filters%22%3A1%2C%22has_search%22%3Atrue%7D&parent=catalog&search_id=98641666